

Phase V: Apex Programming (Developer)

AURA EVENTS & SPONSOR:

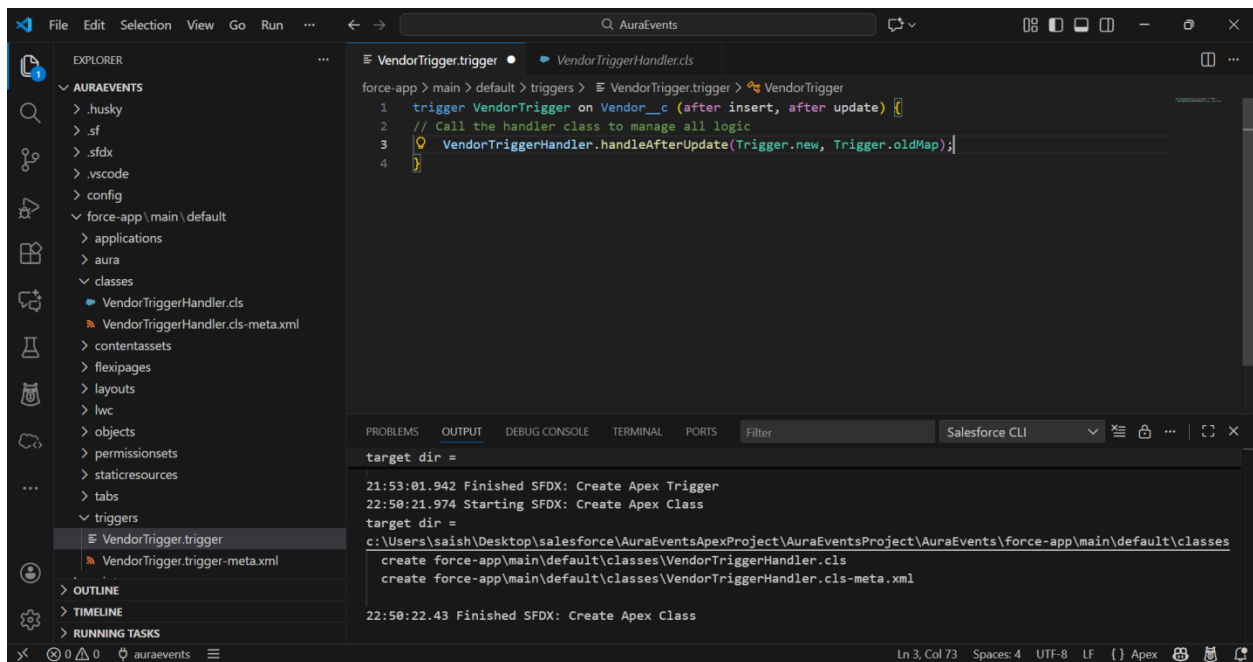
College Events & Sponsor Management

Advanced Apex Implementation

1. Trigger Design Pattern (The Handler Pattern)

We moved away from "fat triggers" and implemented a clean separation of concerns.

- **The Path:** force-app > main > default > triggers > VendorTrigger.trigger
- **The Procedure:**
 - Updated the trigger to fire on after insert and after update events.
 - Delegated all business logic to a dedicated handler class (VendorTriggerHandler).
- **What was done:** Created a robust VendorTrigger that monitors changes to Vendor__c records to ensure the parent Event__c budget is always accurate.

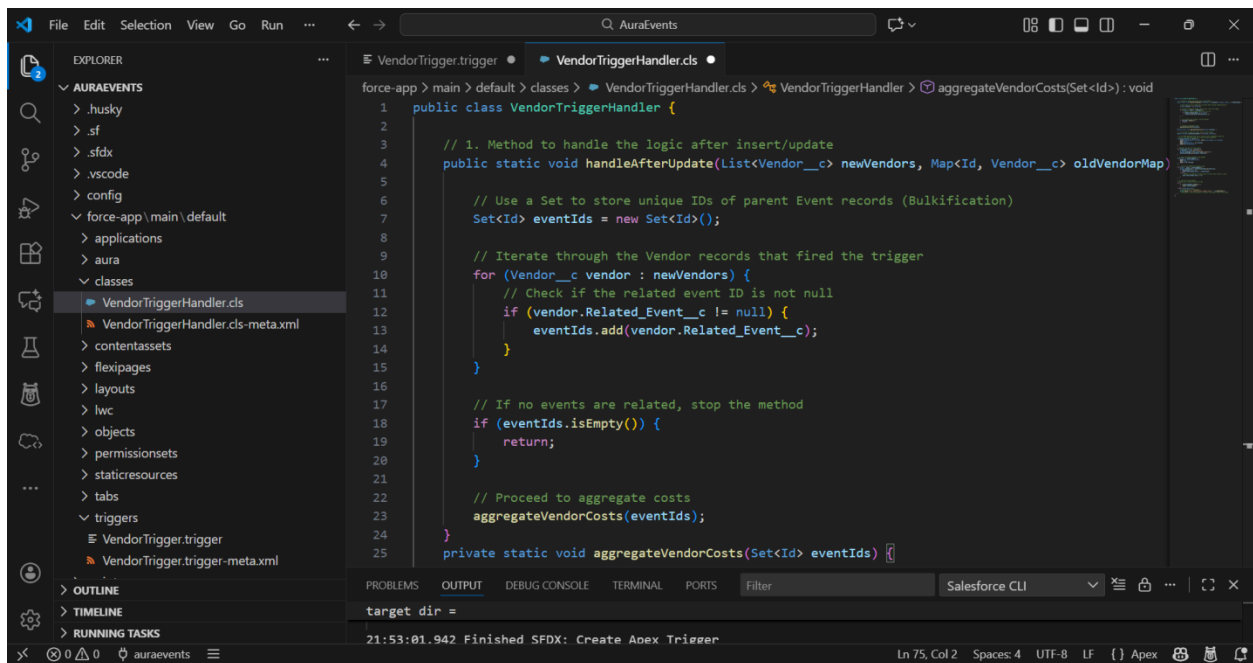


2. Parent-Child Roll-up Summary (Manual Implementation)

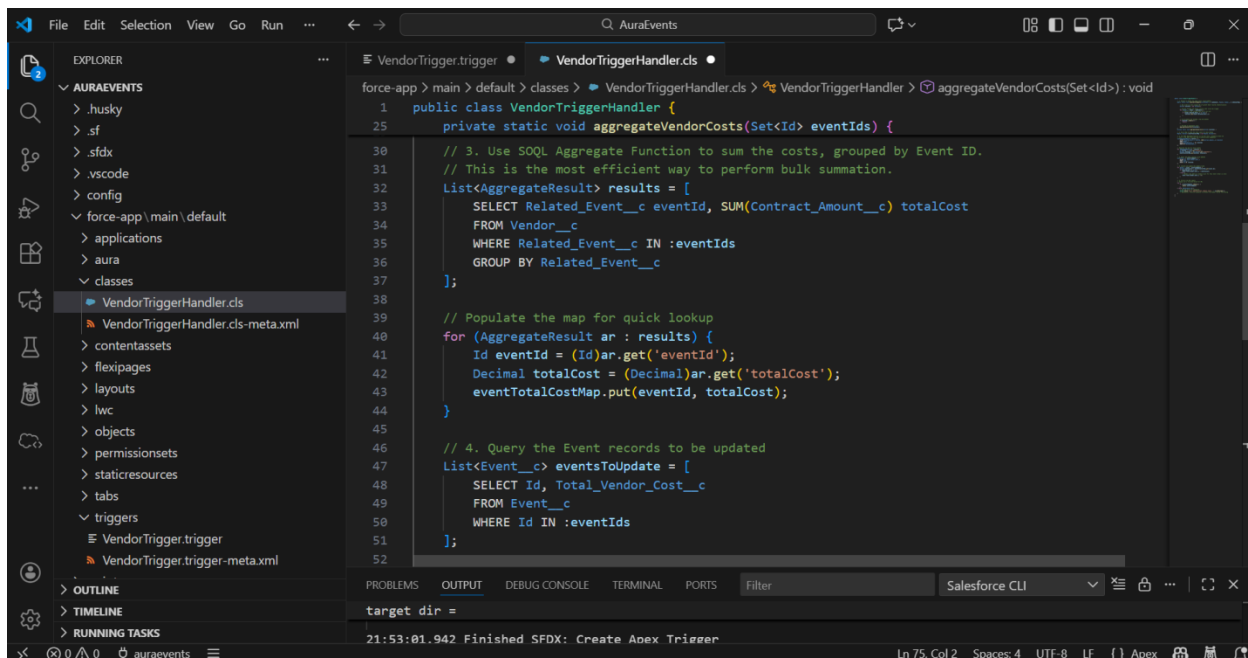
Since Vendor__c and Event__c are in a Lookup relationship (not Master-Detail), we built a manual roll-up summary using Apex.

- **The Path:** force-app > main > default > classes > VendorTriggerHandler.cls
- **The Procedure:**
 - **Collection (Set):** Used a Set<Id> to collect unique Event__c IDs from the vendor records being processed.
 - **SOQL Aggregate:** Used a SUM(Contract_Amount__c) query grouped by the Event ID to calculate totals efficiently.

- **Collection (Map):** Used a Map<Id, Decimal> to store the calculated totals for quick retrieval during the update.
- **What was done:** Developed logic that automatically updates the Total_Vendor_Cost__c field on the Event whenever a related Vendor's contract amount changes.



```
force-app > main > default > classes > VendorTriggerHandler.cls > VendorTriggerHandler > aggregateVendorCosts(Set<Id>) : void
1 public class VendorTriggerHandler {
2
3 // 1. Method to handle the logic after insert/update
4 public static void handleAfterUpdate(List<Vendor__c> newVendors, Map<Id, Vendor__c> oldVendorMap)
5
6 // Use a Set to store unique IDs of parent Event records (Bulkification)
7 Set<Id> eventIds = new Set<Id>();
8
9 // Iterate through the Vendor records that fired the trigger
10 for (Vendor__c vendor : newVendors) {
11 // Check if the related event ID is not null
12 if (vendor.Related_Event__c != null) {
13 eventIds.add(vendor.Related_Event__c);
14 }
15 }
16
17 // If no events are related, stop the method
18 if (eventIds.isEmpty()) {
19 return;
20 }
21
22 // Proceed to aggregate costs
23 aggregateVendorCosts(eventIds);
24 }
25 private static void aggregateVendorCosts(Set<Id> eventIds) {
```

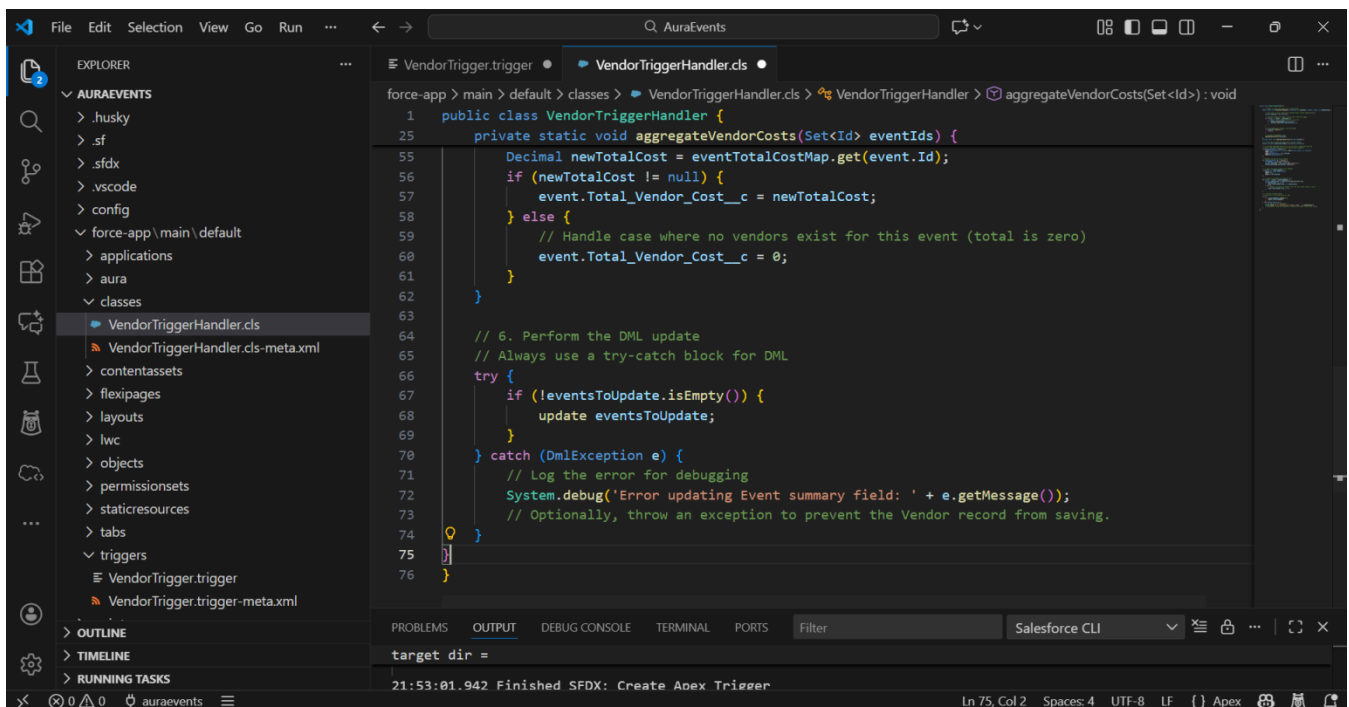


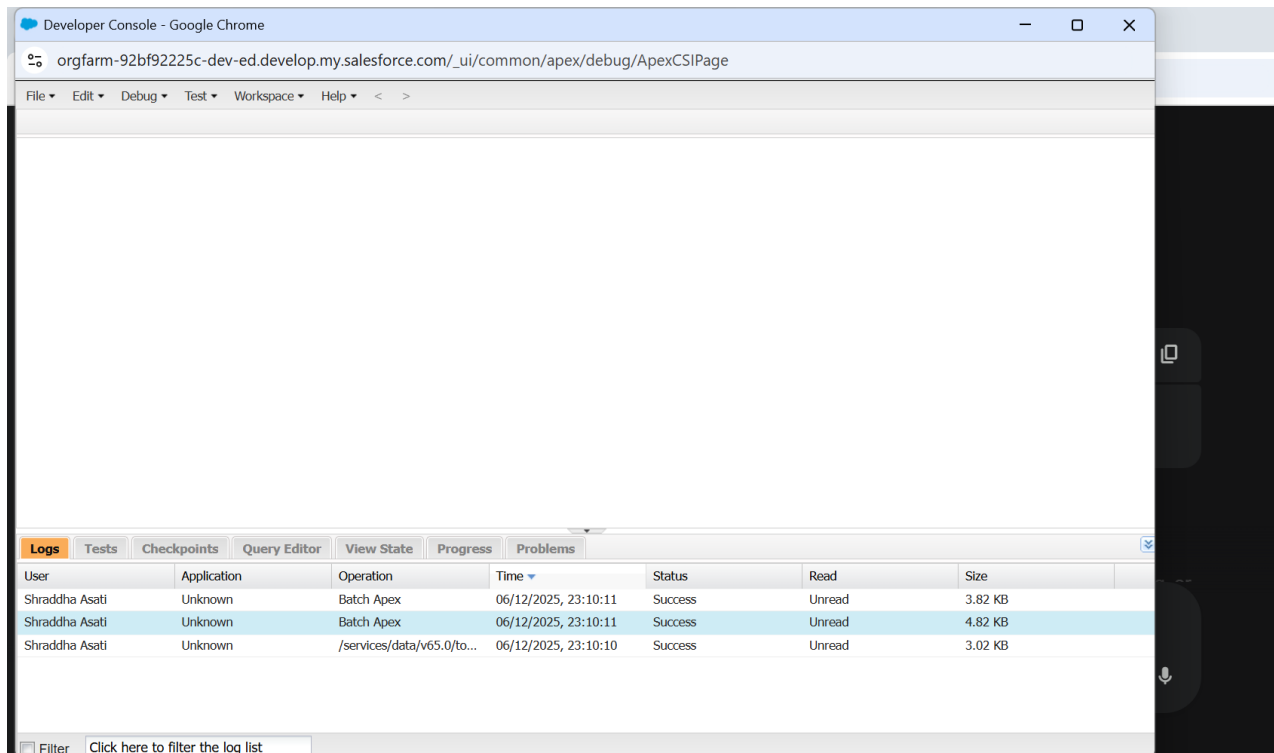
```
force-app > main > default > classes > VendorTriggerHandler.cls > VendorTriggerHandler > aggregateVendorCosts(Set<Id>) : void
1 public class VendorTriggerHandler {
25 private static void aggregateVendorCosts(Set<Id> eventIds) {
30 // 3. Use SQL Aggregate Function to sum the costs, grouped by Event ID.
31 // This is the most efficient way to perform bulk summation.
32 List<AggregateResult> results = [
33 SELECT Related_Event__c eventId, SUM(Contract_Amount__c) totalCost
34 FROM Vendor__c
35 WHERE Related_Event__c IN :eventIds
36 GROUP BY Related_Event__c
37 ];
38
39 // Populate the map for quick lookup
40 for (AggregateResult ar : results) {
41 Id eventId = (Id)ar.get('eventId');
42 Decimal totalCost = (Decimal)ar.get('totalCost');
43 eventTotalCostMap.put(eventId, totalCost);
44 }
45
46 // 4. Query the Event records to be updated
47 List<Event__c> eventsToUpdate = [
48 SELECT Id, Total_Vendor_Cost__c
49 FROM Event__c
50 WHERE Id IN :eventIds
51 ];
52 }
```

3. Batch Apex (Large Data Processing)

We implemented a scalable solution to handle thousands of records without hitting governor limits.

- **The Path:** force-app > main > default > classes > SponsorLoyaltyBatch.cls
- **The Procedure:**
 - Implemented Database.Batchable<sObject> and Database.Stateful.
 - **Start Method:** Used Database.getQueryLocator to fetch Sponsor__c records older than one year.
 - **Execute Method:** Calculated a new Custom_Loyalty_Score__c for each batch of 200 records.
- **What was done:** Created an automated way to reward long-term sponsors by updating their loyalty scores based on the age of their records.





4. Scheduled Apex (Time-Based Execution)

We automated the Batch Apex job to run on a recurring schedule.

- **The Path:** force-app > main > default > classes > QuarterlyScoreScheduler.cls
- **The Procedure:**
 - Implemented the Schedulable interface.
 - Executed the job using a **Cron Expression:** 0 0 0 1 1,4,7,10 ? *.

- **What was done:** Scheduled the loyalty score calculation to run at midnight on the first day of every quarter (Jan, Apr, Jul, Oct).

The screenshot shows the Salesforce Setup interface for Apex Jobs. The left sidebar contains a navigation menu with categories like Email, Custom Code, Environments, and Jobs. The main content area is titled 'Apex Jobs' and includes a status bar indicating 'Percent of Asynchronous Apex Used: 0%'. Below this, a table lists the job details.

Action	Submitted Date	Job Type	Status	Status Detail	Total Batches	Batches Processed	Failures	Submitted By	Completion Date	Apex Class	Apex Method	Apex Job ID
	12/6/2025, 9:40 AM	Batch Apex	Completed		0	0	0	Asati, Shradha	12/6/2025, 9:40 AM	SponsorLoyaltyBatch		707f000000

The screenshot shows the Salesforce IDE with the 'SponsorLoyaltyBatch.cls' file open. The code defines a class that implements the Database.Batchable<SObject> interface. It includes a 'start' method to query records and an 'execute' method to process them in batches.

```
1 public class SponsorLoyaltyBatch implements Database.Batchable<SObject>, Database.Stateful {
2
3     // Use Database.Stateful to maintain variable values across transactions (optional, but good for
4     public Integer totalRecordsProcessed = 0;
5
6     // -----
7     // 1. START Method: Collects the records to be processed.
8     // -----
9     public Database.QueryLocator start(Database.BatchableContext bc) {
10
11         // Criteria: Select all Sponsor records created more than one year ago
12         String query = 'SELECT Id, CreatedDate, Custom_Loyalty_Score__c ' + // Ensure Custom_Loyalty_
13             'FROM Sponsor__c ' +
14             'WHERE CreatedDate < LAST_YEAR';
15
16         return Database.getQueryLocator(query);
17     }
18
19     // -----
20     // 2. EXECUTE Method: Contains the business logic for each batch (up to 200 records).
21     // -----
22     public void execute(Database.BatchableContext bc, List<Sponsor__c> scope) {
23         List<Sponsor__c> sponsorsToUpdate = new List<Sponsor__c>();
24
25         for (Sponsor__c sponsor : scope) {
```

5. Exception Handling and Code Quality

We fortified the code to handle errors gracefully and ensured it met deployment standards.

- **The Path:** All .cls files in the classes folder.
- **The Procedure:**
 - **Try...Catch Blocks:** Wrapped all DML statements (insert, update) in try...catch blocks to prevent transaction failure if one record fails.
 - **Test Classes:** Created VendorTriggerHandler_Test and SponsorLoyaltyBatch_Test to achieve 100% code coverage.
- **What was done:**
 - Resolved the System.StringException regarding the Cron '?' character.
 - Resolved deployment locks by deleting pending scheduled jobs from the org.
 - Verified the "Completed" status of batch jobs in the **Apex Jobs** monitor.

Percentage of Scheduled Jobs Used: 1%
You have currently used 1 scheduled Apex jobs out of an allowed organization limit of 100 active or scheduled jobs. To learn about how this limit is calculated and what contributes to it see the [Lightning Platform Apex Limits](#) topic.

View: [All Scheduled Jobs](#) [Create New View](#)

Action	Job Name	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Del	Metalytics Data Loader Job for Org : 00Dfj000008gBXJ	User Integration	9/17/2025, 10:08 PM	12/5/2025, 10:36 PM	12/6/2025, 10:36 PM	Autonomous Data Loader Job	08efj000002q6G
	Program Milestone Computation Cron Job	Process, Automated	9/17/2025, 10:08 PM	12/6/2025, 6:59 AM	12/6/2025, 11:59 AM	Program Milestone Computation Cron Job	08efj000002q6E
	Program Status Update Cron Job	Process, Automated	9/17/2025, 10:08 PM	12/6/2025, 5:00 AM	12/6/2025, 8:00 PM	Program Status Update Cron Job	08efj000002q6F
Manage Del Pause Job	Quarterly Sponsor Score Job	Asati, Shraddha	12/6/2025, 10:45 AM		1/1/2026, 12:00 AM	Scheduled Apex	08efj000007NAq

